



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY



Semestrální práce

KIV/UPS Síťová hra Oko Bere

Michal Roučka



Obsah

1	Úvod	4
2	Popis hry	5
2.1	Základní pravidla hry Oko Bere	5
2.2	Průběh hry	5
2.3	Karetní balíček	6
3	Popis protokolu	7
3.1	Základní formát zpráv	7
3.2	Konstanty protokolu	7
3.3	Přenášené datové typy	7
3.3.1	Základní datové typy	7
3.3.2	Stavy klienta	8
3.3.3	Stavy místnosti	8
3.4	Příkazy klient → server	8
3.5	Příkazy server → klient	9
3.6	Omezení vstupních hodnot a validace dat	9
3.6.1	Validace nickname	9
3.6.2	Validace zpráv	10
3.6.3	Validace stavu klienta	10
3.6.4	Validace Room Name	10
3.6.5	Validace Room ID	10
3.7	Chybové stavy a jejich hlášení	11
3.7.1	Zpracování chyb	11
3.8	Návaznost zpráv	12
3.8.1	Připojení a přihlášení	12
3.8.2	Vytvoření místnosti	12
3.8.3	Připojení k místnosti a začátek hry	12
3.8.4	Průběh hry	12
3.8.5	Konec kola	13

3.8.6	Konec hry	13
3.8.7	Reconnect scénář	13
3.9	Stavový diagram protokolu	14
4	Popis implementace	15
4.1	Server (C++)	15
4.1.1	Dekompozice do modulů/tříd	15
4.1.2	Rozvrstvení aplikace	15
4.1.3	Použité knihovny a prostředí	16
4.1.4	Metoda paralelizace	17
4.1.5	Reconnect mechanismus	18
4.2	Klient (Java)	19
4.2.1	Dekompozice do modulů/tříd	19
4.2.2	Rozvrstvení aplikace	19
4.2.3	Použité knihovny a prostředí	20
4.2.4	Asynchronní zpracování zpráv	20
4.2.5	Automatický reconnect	21
5	Požadavky na překlad, spuštění a běh	22
5.1	Server (C++)	22
5.1.1	Požadavky	22
5.1.2	Postup překladu	22
5.1.3	Spuštění serveru	23
5.1.4	Ukončení serveru	23
5.2	Klient (Java)	23
5.2.1	Požadavky	23
5.2.2	Postup překladu	23
5.2.3	Spuštění klienta	24
5.2.4	Konfigurace připojení	24
5.3	Testování	24
5.3.1	Spuštění více klientů	24
5.3.2	Valgrind (server)	25
6	Závěr a zhodnocení	26
6.1	Dosažené výsledky	26
6.1.1	Funkční požadavky	26
6.1.2	Technické řešení	26
6.2	Poznatky a zkušenosti	27
6.2.1	Výhody zvoleného přístupu	27
6.2.2	Možná vylepšení	27

6.3 Závěrečné shrnutí	27
Seznam obrázků	28
Seznam tabulek	29

Úvod

1

Tato dokumentace popisuje implementaci síťové multiplayer karetní hry **Oko Bere** v rámci semestrální práce předmětu KIV/UPS. Aplikace využívá architekturu klient-server s TCP komunikací. Server je implementován v jazyce C++ s využitím paralelního zpracování pomocí systémového volání `select()`, klient je realizován v jazyce Java s grafickým rozhraním JavaFX.

Cílem této práce bylo vytvořit funkční síťovou hru s podporou více současně probíhajících her, s robustním protokolem, validací zpráv, a mechanismem automatického znovupřipojení (reconnect) v případě výpadku spojení.

Popis hry

2

2.1 Základní pravidla hry Oko Bere

Oko Bere je česká varianta karetní hry podobná blackjacku. Hra je určena pro **dva hráče**, kteří se snaží dosáhnout hodnoty co nejblíže číslu **21**, aniž by tuto hodnotu překročili.

2.2 Průběh hry

- Hra se skládá z **několika kol**, přičemž pro vítězství je potřeba získat **3 vítězství** (konstanta `SCORE_TO_WIN = 3`).
- Každý hráč na začátku kola obdrží **2 karty** (konstanta `INIT_HAND_SIZE = 2`).
- Hráči mají přiřazené role:
 - **PLAYER** – hráč, který hraje první v daném kole
 - **BANKER** – protihráč (bankéř)
- Role se po každém kole **střídají**.
- Hráč může provést jednu z následujících akcí:
 - **HIT** – vzít si další kartu
 - **STAND** – zůstat, nebrát si další kartu
- Pokud hráč překročí hodnotu 21, je **busted** (prohráva automaticky).
- Po ukončení akcí obou hráčů se vyhodnotí kolo:
 - Vyhrává hráč, který má hodnotu bližší k 21, aniž by ji překročil.
 - Při **stejně hodnotě vyhrává BANKER**.
- **Speciální pravidlo:** Pokud hráč má dvě **esa**, má automaticky hodnotu 21 (**Oko Bere**).

2.3 Karetní balíček

Hra používá standardní **mariášový balíček** (32 karet):

- **Barvy:** Srdce, Kule, Listy, Žaludy
- **Hodnoty karet:**
 - Sedm: 7 bodů
 - Osm: 8 bodů
 - Devět: 9 bodů
 - Deset: 10 bodů
 - Spodek: 1 bod
 - Svršek: 1 bod
 - Král: 2 body
 - Eso: 11 bodů

Balíček je na začátku každého kola **zamíchán** pomocí generátoru náhodných čísel. Pokud během kola dojdou karty, balíček se automaticky resetuje a znovu zamíchá.

Popis protokolu

3

3.1 Základní formát zpráv

Protokol používá textový formát zpráv s následující strukturou:

PRIKAZ | parametr1 | parametr2 | . . . | parametrN \n

- **Delimiter:** znak | (pipe) odděluje jednotlivé části zprávy
- **Message end:** znak \n (newline) ukončuje zprávu
- **Příkaz:** první část zprávy, určuje typ zprávy
- **Parametry:** volitelné parametry příkazu

3.2 Konstanty protokolu

Konstanta	Hodnota
MAX_MESSAGE_SIZE	4096 bytů
MAX_INVALID_MESSAGES	3
RECONNECT_TIMEOUT	30 sekund
CLIENT_TIMEOUT	10 sekund
MAX_PARAMETERS	100
MAX_NICKNAME_LENGTH	20 znaků
MAX_ROOM_NAME_LENGTH	50 znaků

Tabulka 3.1: Konstanty protokolu

3.3 Přenášené datové typy

3.3.1 Základní datové typy

- **String:** textový řetězec v kódování UTF-8
- **Integer:** celé číslo reprezentované jako textový řetězec
- **Card:** karta ve formátu BARVA-HODNOTA (např. SRDCE-ESO)

- **SessionID**: unikátní identifikátor session, UUID formát
- **RoomID**: číselný identifikátor místnosti

3.3.2 Stav klienta

1. **CONNECTED**: klient je připojen, ale není přihlášen
2. **LOBBY**: klient je přihlášen a nachází se v lobby
3. **IN_ROOM**: klient je v místnosti, čeká na druhého hráče
4. **PLAYING**: klient je ve hře

3.3.3 Stav místnosti

1. **WAITING**: místnost čeká na hráče
2. **PLAYING**: probíhá hra
3. **FINISHED**: hra skončila

3.4 Příkazy klient → server

Příkaz	Parametry	Popis
LOGIN	nickname [sessionId]	Přihlášení s nickem, případně reconnect se sessionId
PING	-	Keep-alive zpráva, odesílaná každých 5 sekund
DISCONNECT	-	Oznámení o odpojení
ROOM_LIST	-	Požadavek na seznam místností
CREATE_ROOM	roomName	Vytvoření nové místnosti
JOIN_ROOM	roomId	Připojení k existující místnosti
LEAVE_ROOM	-	Opuštění místnosti
HIT	-	Vzít si kartu
STAND	-	Stát (nebrat kartu)
RECONNECT	sessionId	Pokus o znovupřipojení se session ID
ACK_DEAL_CARDS	-	Potvrzení přijetí rozdaných karet
ACK_ROUND_END	-	Potvrzení přijetí konce kola
ACK_GAME_END	-	Potvrzení přijetí konce hry
ACK_GAME_STATE	-	Potvrzení přijetí stavu hry
RECONNECT_ACCEPT	-	Přijetí reconnect nabídky
RECONNECT_DECLINE	-	Odmítnutí reconnect nabídky

Tabulka 3.2: Příkazy odesílané klientem

3.5 Příkazy server → klient

Příkaz	Parametry	Popis
OK	sessionId	Potvrzení úspěšného přihlášení
ERROR	message	Chybová zpráva
PONG	-	Odpověď na PING
ROOMS	count roomData...	Seznam místností
ROOM	roomId name players state	Detail místnosti
ROOM_CREATED	roomId	Potvrzení vytvoření místnosti
JOINED	roomId opponentNick	Potvrzení připojení k místnosti
GAME_START	opponentNick round	Začátek hry
DEAL_CARDS	cards...	Rozdání karet
GAME_STATE	round yourScore oppScore role	Stav hry
YOUR_TURN	-	Na řadě je hráč
CARD	card	Přidělení nové karty
OPPONENT_ACTION	action [data]	Akce soupeře (HIT/STAND)
ROUND_END	winner yourValue oppValue yourCards oppCards	Konec kola
GAME_END	winner yourScore oppScore	Konec hry
PLAYER_DISCONNECTED	-	Soupeř se odpojil
PLAYER_RECONNECTED	-	Soupeř se znovu připojil
OPPONENT_LEFT	-	Soupeř opustil hru
RECONNECT_QUERY	roomId opponentNick	Dotaz na reconnect

Tabulka 3.3: Příkazy odesílané serverem

3.6 Omezení vstupních hodnot a validace dat

Server provádí důkladnou validaci všech přijatých zpráv:

3.6.1 Validace nickname

- Délka: 1-20 znaků
- Nesmí být prázdný nebo obsahovat pouze whitespace
- Zakázané znaky: |, \n, \r
- Zakázány control characters (0x00-0x1F kromě mezery a tabu, 0x7F)

- Musí obsahovat alespoň jeden non-whitespace znak
- Nickname musí být unikátní (nesmí být již použitý)

3.6.2 Validace zpráv

- Maximální velikost zprávy: 4096 bytů
- Maximální počet parametrů: 100
- Každá zpráva musí končit znakem \n
- Příkaz musí být validní (ze seznamu podporovaných příkazů)
- Počet parametrů musí odpovídat očekávání pro daný příkaz

3.6.3 Validace stavu klienta

- Klient může provádět pouze akce povolené v jeho aktuálním stavu
- Např. HIT a STAND lze provést pouze ve stavu PLAYING
- CREATE_ROOM a JOIN_ROOM lze provést pouze ve stavu LOBBY

3.6.4 Validace Room Name

- Délka: 1-50 znaků
- Nesmí být prázdný

3.6.5 Validace Room ID

- Musí být číslo (integer)
- Místnost s daným ID musí existovat

3.7 Chybové stavy a jejich hlášení

Server rozlišuje následující chybové stavy:

Chyba	Kdy nastává
Invalid nickname	Nickname neprošel validací
Nickname already in use	Nickname již používá jiný klient
Invalid session ID	Session ID neodpovídá uloženému
Session expired	Session vypršela (timeout 30 sekund)
Already logged in	Pokus o přihlášení když je klient již přihlášen
You are not in lobby	Pokus o akci vyžadující lobby stav
Too many rooms	Dosažen limit místností
Invalid name	Název místnosti neprošel validací
Invalid room ID	Room ID není číslo nebo není validní
Room does not exist	Místnost s daným ID neexistuje
Room is full	Místnost je plná (2 hráči)
Game already in progress	Nelze připojit, hra již běží
You are not in a room	Pokus o akci vyžadující být v místnosti
You are not in game	Pokus o herní akci mimo hru
Invalid parameter count	Nesprávný počet parametrů
Invalid command	Neznámý příkaz
Empty message	Přijata prázdná zpráva
Unable to parse message	Zprávu nelze parsovat
Message too large	Zpráva překračuje MAX_MESSAGE_SIZE
Too many parameters	Více než MAX_PARAMETERS
Reconnect failed	Reconnect selhal
Room no longer exists	Místnost již neexistuje
Already removed	Klient již byl odebrán

Tabulka 3.4: Chybové stavy

3.7.1 Zpracování chyb

- Při každé chybě server odešle zprávu `ERROR|message`
- Každá nevalidní zpráva zvyšuje počítadlo `invalidMessageCount`
- Po **3 nevalidních zprávách** je klient automaticky odpojen
- Timeouty:
 - Klient timeout: 10 sekund (musí poslat PING každých 5 sekund)
 - Reconnect timeout: 30 sekund (čas na znovupřipojení)

3.8 Návaznost zpráv

3.8.1 Připojení a přihlášení

Klient -> Server: LOGIN|nickname
Server -> Klient: OK|sessionId
nebo
Server -> Klient: ERROR|reason

3.8.2 Vytvoření místnosti

Klient -> Server: CREATE_ROOM|roomName
Server -> Klient: ROOM_CREATED|roomId
Server -> Klient: JOINED|roomId|
nebo
Server -> Klient: ERROR|reason

3.8.3 Připojení k místnosti a začátek hry

Klient -> Server: JOIN_ROOM|roomId
Server -> Klient: JOINED|roomId|opponent
Server -> Oba klienti: GAME_START|opponent|round
Server -> Oba klienti: GAME_STATE|round|score|oppScore|role
Klient -> Server: ACK_GAME_STATE
Server -> Oba klienti: DEAL_CARDS|card1,card2
Klient -> Server: ACK_DEAL_CARDS
Server -> Klient: YOUR_TURN

3.8.4 Průběh hry

Klient -> Server: HIT
Server -> Klient: CARD|card
Server -> Opponent: OPPONENT_ACTION|HIT
nebo
Klient -> Server: STAND
Server -> Opponent: OPPONENT_ACTION|STAND
Server -> Opponent: YOUR_TURN

3.8.5 Konec kola

```
Server -> Oba klienti: ROUND_END|winner|yourValue|oppValue|yourCards
Klient -> Server: ACK_ROUND_END
Server -> Oba klienti: GAME_STATE|round|score|oppScore|role
Klient -> Server: ACK_GAME_STATE
Server -> Oba klienti: DEAL_CARDS|card1,card2
...
```

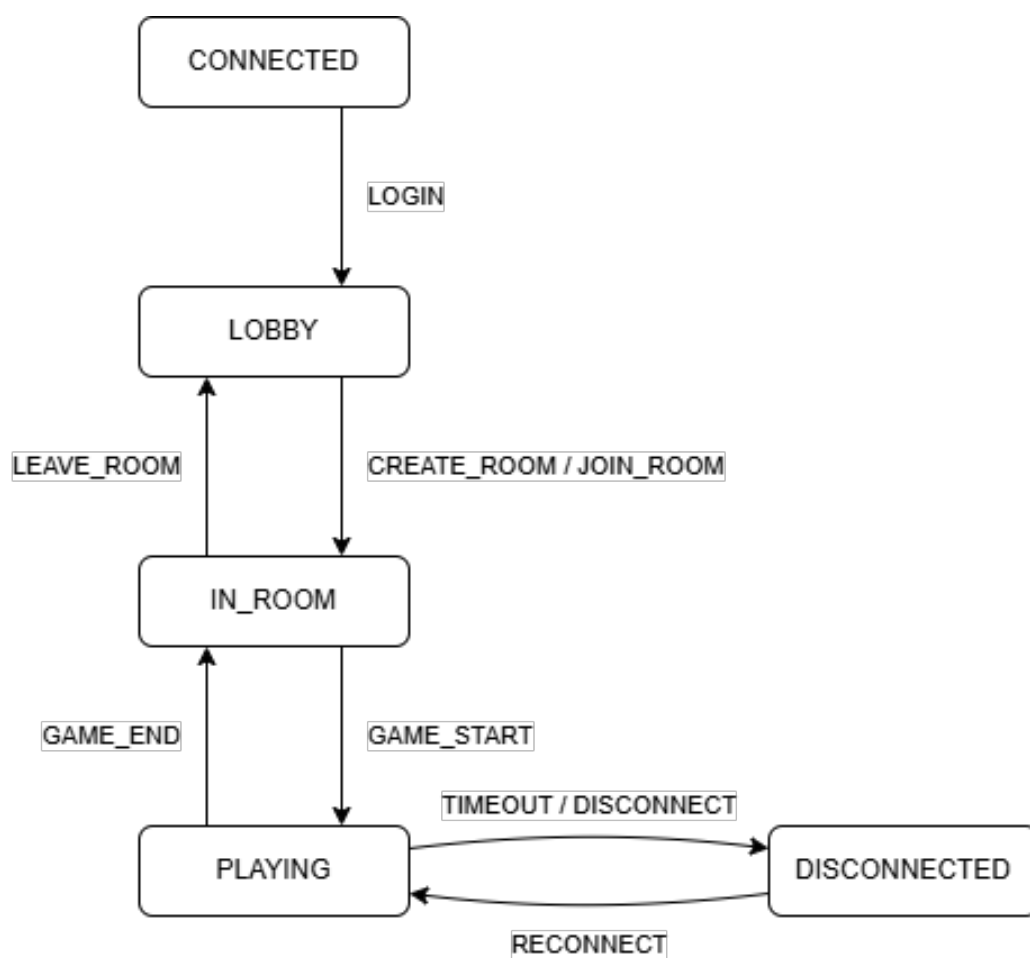
3.8.6 Konec hry

```
Server -> Oba klienti: GAME_END|winner|yourScore|oppScore
Klient -> Server: ACK_GAME_END
```

3.8.7 Reconnect scénář

```
Klient -> Server: LOGIN|nickname|sessionId
Server -> Klient: RECONNECT_QUERY|roomId|opponentNick
Klient -> Server: RECONNECT_ACCEPT
Server -> Klient: OK|sessionId
Server -> Klient: JOINED|roomId|opponent
Server -> Oba klienti: PLAYER_RECONNECTED
Server -> Klient: GAME_STATE|...
Server -> Klient: DEAL_CARDS|...
nebo
Klient -> Server: RECONNECT_DECLINE
Server -> Klient: OK|sessionId
```

3.9 Stavový diagram protokolu



Obrázek 3.1: Stavový diagram klientských stavů

Popis implementace

4

4.1 Server (C++)

4.1.1 Dekompozice do modulů/tříd

Server je strukturován do následujících modulů:

Modul/Třída	Odpovědnost
Server	Hlavní třída serveru, správa socketů, select() loop, routing zpráv
Client	Reprezentace připojeného klienta, buffery, stav, validace
Room	Správa herních místností, max 2 hráči, stavy místnosti
Game	Implementace herní logiky Oko Bere, pravidla, vyhodnocování
Card/Deck	Reprezentace karet a balíčku, míchání, lízání karet
Protocol	Konstanty protokolu, parsování a sestavování zpráv, validace
Logger	Logování událostí serveru do souboru a konzole
main.cpp	Entry point, parsování argumentů, inicializace serveru

Tabulka 4.1: Moduly serveru

4.1.2 Rozvrstvení aplikace

1. Síťová vrstva (Server):

- Správa TCP socketů
- select() pro multiplexing I/O operací
- Příjem a odesílání zpráv
- Buffering (čtecí a zapisovací fronty)

2. Protokolová vrstva (Protocol):

- Parsování zpráv
- Validace formátu zpráv

- Sestavování odpovědí

3. **Aplikační logika** (Server, Room):

- Správa klientů a sessions
- Správa místností
- Routing zpráv mezi klienty
- Timeout handling
- Reconnect mechanismus

4. **Herní logika** (Game):

- Implementace pravidel Oko Bere
- Správa herního stavu
- Rozdávání karet
- Vyhodnocování kol a celkového vítězství

4.1.3 Použité knihovny a prostředí

- **Jazyk:** C++17
- **Kompilátor:** g++ (GCC)
- **Build systém:** GNU Make, CMake
- **Síťové knihovny:** BSD sockets (POSIX)
 - `sys/socket.h` – socket API
 - `sys/select.h` – `select()` multiplexing
 - `netinet/in.h` – IPv4 struktury
 - `arpa/inet.h` – `inet_pton()`, `inet_ntop()`
- **Threading:** pthread (POSIX threads)
- **Standardní knihovna STL:**
 - `std::map`, `std::vector`, `std::string`
 - `std::unique_ptr` – smart pointery
 - `std::mutex` – mutexové zámky pro thread-safety
 - `std::random` – generátor náhodných čísel

Compiler flags:

`CXXFLAGS = -std=c++17 -Wall -Wextra -g -pthread`

4.1.4 Metoda paralelizace

Server používá **synchronní I/O multiplexing pomocí select()**:

- **Hlavní smyčka:** používá `select()` pro čekání na události na více socketech současně
- **Jeden proces:** server běží v jediném procesu
- **Thread-safety:** kritické sekce v Game jsou chráněny mutexem
- **Výhody:**
 - Jednoduchá implementace
 - Nízká režie (žádné přepínání kontextu mezi vlákny)
 - Deterministické chování
- **Omezení:**
 - Škálování limitováno jedním jádrem CPU

Průběh hlavní smyčky:

```
while (running) {
    // Příprava fd_set pro čtení a zápis
    FD_ZERO(&readfds);
    FD_ZERO(&writefds);
    FD_SET(serverSocket, &readfds);

    for (const auto& [socket, client] : clients) {
        FD_SET(socket, &readfds);
        if (client->hasMessagesToSend()) {
            FD_SET(socket, &writefds);
        }
    }

    // Select s timeoutem 1 sekunda
    select(maxfd + 1, &readfds, &writefds, nullptr,
        &timeout);

    // Zpracování nových připojení
    if (FD_ISSET(serverSocket, &readfds)) {
        acceptNewClient();
    }
}
```

```
}

// Zpracovani dat od klientu
for (auto& [socket, client] : clients) {
    if (FD_ISSET(socket, &readfds)) {
        handleClientData(client);
    }
    if (FD_ISSET(socket, &writefds)) {
        sendToClient(client, ...);
    }
}

// Cleanup timeouts
cleanupTimedOutClients();
}
```

4.1.5 Reconnect mechanismus

Server implementuje robustní mechanismus znovupřipojení:

1. Detekce odpojení:

- Timeout (10 sekund bez zpráv)
- Uzavření socketu klientem
- Chyba při čtení/zápisu

2. Uložení stavu:

- Server uloží informace o odpojeném hráči do `disconnectedPlayers`
- Uloží se: `nickname`, `roomId`, `sessionId`, čas odpojení
- Hra zůstává zachována (neruší se)

3. Timeout okno: 30 sekund na znovupřipojení

4. Znovupřipojení:

- Klient pošle `LOGIN|nickname|sessionId`
- Server najde hráče v `disconnectedPlayers`
- Validuje `sessionId`
- Pokud je validní, pošle `RECONNECT_QUERY`
- Klient přijme/odmítne pomocí `RECONNECT_ACCEPT/DECLINE`
- Server obnoví stav hry a pošle aktuální `GAME_STATE` a `DEAL_CARDS`

4.2 Klient (Java)

4.2.1 Dekompozice do modulů/tříd

Třída	Odpovědnost
Launcher	Entry point aplikace, spouští JavaFX
HelloApplication	JavaFX Application třída, načítá FXML
GameController	Hlavní kontroler UI, zpracování událostí, správa stavů
GameClient	High-level herní klient, login, room management, herní akce
NetworkClient	Low-level síťová komunikace, TCP socket, send/receive
ProtocolMessage	Parsování a sestavování protokolových zpráv
MessageValidator	Validace zpráv na straně klienta
CardImageLoader	Načítání a cachování obrázků karet
RoomInfo	Datová třída pro informace o místnosti
Logger	Logování událostí klienta

Tabulka 4.2: Moduly klienta

4.2.2 Rozvrstvení aplikace

1. Prezentační vrstva (JavaFX):

- FXML layout (game-view.fxml)
- GameController – kontroler UI
- Tři hlavní panely: Connection, Lobby, Game
- Vizualizace karet, skóre, akcí

2. Aplikační logika (GameClient):

- Správa herního stavu
- Zpracování příkazů
- Automatický reconnect
- Keep-alive (PING/PONG)

3. Síťová komunikace (NetworkClient):

- TCP Socket komunikace
- BufferedReader/PrintWriter pro textový protokol
- Timeout handling

4. Protokol (ProtocolMessage):

- Parsování zpráv

- Sestavování zpráv
- Factory metody pro běžné zprávy

4.2.3 Použité knihovny a prostředí

- **Jazyk:** Java 21
- **Build systém:** Apache Maven 3.x
- **GUI framework:** JavaFX 21.0.6
 - `javafx-controls` – UI komponenty
 - `javafx-fxml` – FXML loader
- **Síťová komunikace:** Java Socket API
 - `java.net.Socket`
 - `java.io.BufferedReader`
 - `java.io.PrintWriter`
- **Multithreading:** `javafx.concurrent.Task`

4.2.4 Asynchronní zpracování zpráv

Klient používá **asynchronní zpracování** pro příjem zpráv ze serveru:

- **Hlavní vlákno (JavaFX Application Thread):**
 - Zpracovává UI události
 - Renderuje grafiku
- **Síťové vlákno (Task<Void>):**
 - Běží na pozadí
 - Čeká na zprávy ze serveru
 - Parsuje zprávy
 - Aktualizuje UI pomocí `Platform.runLater()`

Příklad asynchronního zpracování:

```
Task<Void> messageTask = new Task<>() {  
    @Override  
    protected Void call() {  
        while (!isCancelled()) {  
            ProtocolMessage msg =  
                gameClient.receiveMessage();  
            if (msg != null) {
```

```
Platform.runLater(() ->
    handleMessage(msg));
    }
}
return null;
}
};
new Thread(messageTask).start();
```

4.2.5 Automatický reconnect

Klient implementuje automatické znovupřipojení:

1. Detekce odpojení:

- Exception při čtení/zápisu
- Timeout socketu
- Null zpráva od serveru

2. Pokus o reconnect:

- Klient uloží nickname a sessionId
- Pokusí se znovu připojit k serveru
- Pošle LOGIN|nickname|sessionId

3. Reconnect dialog:

- Pokud server pošle RECONNECT_QUERY, zobrazí se dialog
- Uživatel může přijmout nebo odmítnout
- Při přijetí se hra obnoví v aktuálním stavu

Požadavky na překlad, spuštění a běh

5

5.1 Server (C++)

5.1.1 Požadavky

- **OS:** Linux
- **Kompilátor:** g++ s podporou C++17 nebo novější
- **Build nástroje:** GNU Make nebo CMake 3.10+
- **Knihovny:** pthread

5.1.2 Postup překladu

Varianta 1: Makefile

```
cd server  
make
```

Výsledný binární soubor: ./oko_bere_server

Varianta 2: CMake

```
cd server  
mkdir build  
cd build  
cmake ..  
make
```

Výsledný binární soubor: ./build/oko_bere_server

5.1.3 Spuštění serveru

Základní spuštění:

```
./ oko_bere_server <IP> <PORT>
```

S parametry:

```
./ oko_bere_server <IP> <PORT> -c <MAX_CLIENTS> -r <MAX_ROOMS>
```

Příklad:

```
./ oko_bere_server 127.0.0.1 10000 -c 20 -r 10
```

Pomocí Makefile:

```
make run IP=127.0.0.1 PORT=10000 C=20 R=10
```

Parametry:

- **IP:** IP adresa, na které server naslouchá (např. 127.0.0.1, 0.0.0.0)
- **PORT:** Port (např. 10000)
- **-c MAX_CLIENTS:** Maximální počet klientů (výchozí 10)
- **-r MAX_ROOMS:** Maximální počet místností (výchozí 5)

5.1.4 Ukončení serveru

Server lze ukončit pomocí Ctrl+C (SIGINT). Server provede graceful shutdown:

- Odpojí všechny klienty
- Uvolní všechny prostředky
- Zavře všechny sockety

5.2 Klient (Java)

5.2.1 Požadavky

- **JDK:** Java 21 nebo novější
- **Build nástroj:** Apache Maven 3.x nebo přiložený Maven Wrapper
- **JavaFX:** verze 21.0.6 (automaticky staženo Mavenem)

5.2.2 Postup překladu

Varianta 1: Maven:

```
cd client
mvn clean compile
```


Varianta 2: Maven Wrapper (nevyžaduje instalaci Mavenu):

```
cd client
./mvnw clean compile
```

Maven Wrapper automaticky stáhne a použije správnou verzi Mavenu při prvním spuštění.

5.2.3 Spuštění klienta

Pomocí Maven:

```
cd client
mvn clean javafx:run
```

Pomocí Maven Wrapper (nevyžaduje instalaci Mavenu):

```
cd client
./mvnw clean javafx:run
```

5.2.4 Konfigurace připojení

Po spuštění klienta:

1. Zadejte IP adresu serveru (např. 127.0.0.1)
2. Zadejte port serveru (např. 10000)
3. Zadejte nickname (1-20 znaků, bez speciálních znaků)
4. Klikněte na "Connect"

5.3 Testování

5.3.1 Spuštění více klientů

Pro testování hry je potřeba spustit alespoň 2 klienty:

```
# Terminal 1
cd client
mvn clean javafx:run
```

```
# Terminal 2
cd client
mvn clean javafx:run
```

5.3.2 Valgrind (server)

Pro kontrolu paměťových úniků:

```
cd server  
make valgrind
```

Závěr a zhodnocení

6

6.1 Dosažené výsledky

V rámci této semestrální práce byl úspěšně implementován kompletní systém pro síťovou multiplayer karetní hru Oko Bere. Výsledná aplikace splňuje všechny požadavky zadání:

6.1.1 Funkční požadavky

- **Implementace hry:** Hra Oko Bere je plně funkční s kompletními pravidly
- **Multiplayer:** Podpora více současně probíhajících her v různých místnostech
- **Síťová komunikace:** Spolehlivý TCP protokol s validací zpráv
- **Reconnect mechanismus:** Automatické znovupřipojení po výpadku spojení
- **Grafické rozhraní:** Intuitivní JavaFX GUI s vizualizací karet

6.1.2 Technické řešení

- **Server:** Efektivní implementace v C++ s použitím `select()` pro paralelní obsluhu
- **Klient:** Moderní Java aplikace s asynchronním zpracováním zpráv
- **Protokol:** Dobře navržený textový protokol s důkladnou validací
- **Bezpečnost:** Ochrana proti útokům (buffer overflow, nevalidní zprávy)
- **Robustnost:** Timeout handling, error recovery, graceful shutdown

6.2 Poznatky a zkušenosti

6.2.1 Výhody zvoleného přístupu

- **Select() multiplexing:** Jednoduchá implementace, nízká režie, deterministické chování
- **Textový protokol:** Snadné debugování, čitelnost, jednoduchost implementace
- **Stavový automat:** Přehledná správa stavů klienta a místností
- **JavaFX:** Moderní, výkonné GUI s jednoduchým FXML layoutem

6.2.2 Možná vylepšení

- **Binární protokol:** Pro vyšší výkon by bylo možné použít binární protokol
- **Šifrování:** Přidání TLS/SSL pro zabezpečenou komunikaci
- **Persistence:** Uložení herních dat do databáze pro statistiky
- **Chat:** Implementace textového chatu mezi hráči
- **AI opponent:** Možnost hrát proti počítači

6.3 Závěrečné shrnutí

Projekt byl úspěšně dokončen a všechny požadavky zadání byly splněny. Aplikace je plně funkční, testovaná a připravená k použití. Implementace prokázala schopnost navrhnout a realizovat robustní síťovou aplikaci s využitím moderních technologií a osvědčených návrhových vzorů.

Práce na projektu poskytla cenné zkušenosti v oblasti:

- Návrhu síťových protokolů
- Paralelního programování (select(), asynchronní zpracování)
- Programování v C++ a Java
- Tvorby GUI aplikací s JavaFX
- Debugování síťových aplikací
- Řešení problémů s timeouty a reconnect mechanikou

Výsledná aplikace je stabilní, dobře strukturovaná a připravená pro případné budoucí rozšíření.

Seznam obrázků

3.1	Stavový diagram klientských stavů	14
-----	---	----

Seznam tabulek

3.1	Konstanty protokolu	7
3.2	Příkazy odesílané klientem	8
3.3	Příkazy odesílané serverem	9
3.4	Chybové stavy	11
4.1	Moduly serveru	15
4.2	Moduly klienta	19